

**LAPORAN PRAKTIKUM
STRUKTUR DATA
“*QUEUE* DAN *LINKEDLIST*”
PADA BAHASA PEMROGRAMAN JAVA**

disusun Oleh :

RAZIF AL FARISI

2511532028

Dosen Pengampu : Dr. Wahyudi S.T, M.T

Asisten Laboratorium : Sherly Sukmadira



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2026**

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum yang berjudul “*Queue pada Bahasa Pemrograman Java*” dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum pada mata kuliah Struktur Data, khususnya mengenai penggunaan struktur data *Queue* dalam bahasa pemrograman Java. Praktikum ini bertujuan untuk memberikan pemahaman kepada mahasiswa mengenai konsep struktur data *FIFO (First In First Out)* dan operasi dasar dari *Queue* tersebut seperti enqueue, dequeue, dan reverse data, serta implementasinya dalam program.

Penulis menyadari bahwa penyusunan laporan ini tidak terlepas dari bimbingan, bantuan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data yang telah memberikan ilmu dan arahan selama proses pembelajaran.

Padang, 30 April 2026

Penulis

Daftar Isi

KATA PENGANTAR	i
BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan	3
BAB II	4
PEMBAHASAN	4
2.1 Landasan Teori	4
2.2 Implementasi Program	4
2.2.1 Class Mahasiswa	4
2.2.2 Class Mahasiswa Driver	8
2.2.3 Class DaftarKata	10
2.2.4 Class DaftarKataDriver.....	12
2.3 Tugas Praktikum	15
2.3.1 Class Musik	15
2.3.2 Class Playlist (Driver Musik)	20
BAB III.....	26
PENUTUP.....	26
3.1 Kesimpulan.....	26
3.2 Saran	26
Referensi.....	27

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pemrograman yang digunakan untuk mengatur dan mengelola data secara efisien. Salah satu struktur data yang sering digunakan adalah queue (antrian), yang bekerja dengan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar.

Konsep queue banyak digunakan dalam kehidupan sehari-hari, seperti antrian di loket pelayanan, rumah sakit, maupun dalam sistem komputer seperti proses scheduling. Oleh karena itu, pemahaman mengenai queue sangat penting bagi mahasiswa dalam mempelajari struktur data.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami cara kerja queue serta mengimplementasikannya menggunakan berbagai metode.

1.2 Tujuan

Tujuan dari penyusunan laporan praktikum ini adalah sebagai berikut:

1. Memahami konsep *Queue* dalam struktur data
2. Mengimplementasikan *Queue* menggunakan array dan *LinkedList*
3. Memahami operasi dasar *Queue* seperti *enqueue*, *dequeue*, dan lainnya
4. Mengimplementasikan *Queue* dan *Stack* untuk membuat method *Reverse* data

BAB II

PEMBAHASAN

2.1 Landasan Teori

Queue (antrian) merupakan salah satu struktur data linear yang menggunakan prinsip *FIFO (First In First Out)*, yaitu elemen yang pertama kali masuk ke dalam antrian akan menjadi elemen pertama yang keluar. Konsep ini menyerupai antrian dalam kehidupan sehari-hari, seperti antrian di loket pembayaran, di mana orang yang datang lebih dulu akan dilayani terlebih dahulu.

Dalam implementasinya, queue memiliki dua operasi utama, yaitu :

- **Enqueue**, yaitu proses menambahkan elemen ke bagian belakang (*rear*) antrian.
- **Dequeue**, yaitu proses menghapus elemen dari bagian depan (*front*) antrian.

Selain itu, terdapat beberapa operasi tambahan yang sering digunakan, antara lain :

- **Peek/Front** → untuk melihat elemen paling depan tanpa menghapusnya
- **Rear** → untuk melihat elemen paling belakang
- **isEmpty** → untuk mengecek apakah antrian kosong
- **isFull** → untuk mengecek apakah antrian penuh (khusus implementasi array)

Queue juga memiliki beberapa jenis seperti *Linear Queue*, *Circular Queue*, *Priority*, dan *Deque (Double Ended Queue)*. Pada praktikum kali ini jenis *Queue* yang digunakan adalah *Circular Queue*.

2.2 Implementasi Program

Berikut adalah beberapa implementasi *Queue* :

2.2.1 Class QueueArray_2511532028

```
package pekan4_2511532028;

public class QueueArray_2511532028 {
    int front_2028, rear_2028, size_2028;
    int capacity_2028;
    int array_2028[];

    public QueueArray_2511532028(int capacity) {
        this.capacity_2028 = capacity;
        front_2028 = this.size_2028 = 0;
        rear_2028 = capacity - 1;
    }
}
```

```

        array_2028 =new int[this.capacity_2028];
    }

    boolean isFull(QueueArray_2511532028 queue) {
        return (queue.size_2028 == queue.capacity_2028);
    }

    boolean isEmpty(QueueArray_2511532028 queue) {
        return (queue.size_2028 == 0);
    }

    void enqueue_2028(int item_2028) {
        if(isFull(this)) {
            return;
        }
        this.rear_2028 = (this.rear_2028 + 1) % this.capacity_2028;
        this.array_2028[this.rear_2028] = item_2028;
        this.size_2028 = this.size_2028 + 1;
        System.out.println(item_2028 + " enqueue to queue");
    }

    int dequeue_2028() {
        if(isEmpty(this))
            return Integer.MIN_VALUE;
        int item = this.array_2028[this.front_2028];
        this.front_2028 = (this.front_2028 + 1) % this.capacity_2028;
        this.size_2028 = this.size_2028 - 1;
        return item;
    }

    int front_2028() {
        if(isEmpty(this))
            return Integer.MIN_VALUE;
        return this.array_2028[this.front_2028];
    }

    int rear_2028() {
        if(isEmpty(this))
            return Integer.MIN_VALUE;
        return this.array_2028[this.rear_2028];
    }

```

```

    }

    //mencetak elemen antrian
    void display_2028() {
        int i;
        if (front_2028 == rear_2028) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }
        //kunjungi dari belakang dan cetak
        for (i = front_2028; i < rear_2028; i++) {
            System.out.printf(" %d <-- ", array_2028[i]);
        }
        return;
    }
}

```

Penjelasan :

Class ini merupakan implementasi *Queue* menggunakan array.

```

public QueueArray_2511532028(int capacity) {
    this.capacity_2028 = capacity;
    front_2028 = this.size_2028 = 0;
    rear_2028 = capacity - 1;
    array_2028 = new int[this.capacity_2028];
}

```

Kode ini disebut sebagai Konstruktor, yaitu untuk menginisialisasi *Queue*. Variabel *capacity* menentukan kapasitas maksimum, sedangkan *front*, *rear*, dan *size* digunakan untuk mengatur posisi elemen dalam *Queue*.

```

boolean isFull(QueueArray_2511532028 queue) {
    return (queue.size_2028 == queue.capacity_2028);
}

boolean isEmpty(QueueArray_2511532028 queue) {
    return (queue.size_2028 == 0);
}

```

Method ini adalah untuk melakukan pengecekan, apakah tempat penyimpanan data *Queue* (Array) tadi sudah penuh atau masih kosong.

```

void enqueue_2028(int item_2028) {
    if(isFull(this)) {

```

```

        return;
    }
    this.rear_2028 = (this.rear_2028 + 1) % this.capacity_2028;
    this.array_2028[this.rear_2028] = item_2028;
    this.size_2028 = this.size_2028 + 1;
    System.out.println(item_2028 + " enqueue to queue");
}

```

Method ini disebut *Enqueue* yang berfungsi untuk menambahkan elemen ke dalam *Queue*. Saat *Queue* sudah penuh, method ini tidak akan dijalankan. Jika tidak penuh, posisi *rear* akan bergeser ke posisi berikutnya. Pada *rear* ini diterapkan *Circular Queue*. Lalu elemen di tambahkan ke dalam *Queue* pada posisi *rear*. *Size Queue* bertambah satu karena sudah berisikan elemen yang tadi di tambahkan.

```

int dequeue_2028() {
    if(isEmpty(this))
        return Integer.MIN_VALUE;
    int item = this.array_2028[this.front_2028];
    this.front_2028 = (this.front_2028 + 1) % this.capacity_2028;
    this.size_2028 = this.size_2028 - 1;
    return item;
}

```

Method ini digunakan untuk menghapus nilai terdepan (*front*). Jika *Queue* tidak berisi elemen (*isEmpty*), method ini akan mengembalikan nilai terkecil dari tipe data tersebut. Jika *Queue* tidak kosong, elemen di dalam *Queue* pada posisi *front* akan di hapus, setelah itu posisi *front* bergeser (+1). Dengan *Circular Queue*, saat posisi *front* sama dengan kapasitas dari *Queue*, posisi *front* kembali ke awal (posisi 0). *Size* atau data dalam *Queue* berkurang (-1).

```

int front_2028() {
    if(isEmpty(this))
        return Integer.MIN_VALUE;
    return this.array_2028[this.front_2028];
}

int rear_2028() {
    if(isEmpty(this))
        return Integer.MIN_VALUE;
    return this.array_2028[this.rear_2028];
}

```



```
}
```

Method ini hanya melihat nilai terdepan dan paling belakang.

```
void display_2028() {
    int i;
    if (front_2028 == rear_2028) {
        System.out.printf("\nAntrian Kosong\n");
        return;
    }
    //kunjungi dari belakang dan cetak
    for (i = front_2028; i < rear_2028; i++) {
        System.out.printf(" %d <-- ", array_2028[i]);
    }
    return;
}
}
```

Method ini menampilkan setiap data yang sudah di tambahkan menggunakan *enqueue()* menggunakan perulangan for dengan i bernilai sama dengan *front*.

Variable :

2.2.2 Class QueueArrayDriver_2511532028

```
package pekan4_2511532028;

public class QueueArrayDriver_2511532028 {

    public static void main(String[] args) {
        QueueArray_2511532028 queue_2028 = new QueueArray_2511532028(1000);
        queue_2028.enqueue_2028(10);
        queue_2028.enqueue_2028(20);
        queue_2028.enqueue_2028(30);
        queue_2028.enqueue_2028(40);
        System.out.println("item di depan " + queue_2028.front_2028);
        System.out.println("item paling belakang " + queue_2028.rear_2028);
        System.out.println("tampilan queue");
        queue_2028.display_2028();
        System.out.println();
        System.out.println(queue_2028.dequeue_2028() + " dihapus dari queue");

        System.out.println("item di depan : " + queue_2028.front_2028);
    }
}
```

```

        System.out.println("item paling belakang : " +
queue_2028.rear_2028);
        System.out.println("tampilan queue setelah satu data dihapus");
        queue_2028.display_2028();

    }

}

```

Penjelasan :

```
QueueArray_2511532028 queue_2028 = new QueueArray_2511532028(1000);
```

Membuat objek *Queue* dengan kapasitas (1000).

```

queue_2028.enqueue_2028(10);
queue_2028.enqueue_2028(20);
queue_2028.enqueue_2028(30);
queue_2028.enqueue_2028(40);

```

Menambahkan elemen ke dalam *Queue* secara berurutan.

```

System.out.println("item di depan " + queue_2028.front_2028);
System.out.println("item paling belakang " + queue_2028.rear_2028);

```

Untuk menampilkan elemen terdepan dan paling belakang.

```

System.out.println("tampilan queue");
queue_2028.display_2028();

```

Menampilkan isi dari *Queue*.

```
System.out.println(queue_2028.dequeue_2028() + " dihapus dari queue");
```

Untuk menghapus elemen dan menampilkan elemen terdepan yang sudah di hapus.

Gambar 2.2.2

```
<terminated> QueueArrayDriver_2511532028 [Java Application] C:\Users\USER\
10 enqueue to queue
20 enqueue to queue
30 enqueue to queue
40 enqueue to queue
item di depan 0
item paling belakang 3
tampilan queue
 10 <-- 20 <-- 30 <--
10 dihapus dari queue
item di depan : 1
item paling belakang : 3
tampilan queue setelah satu data dihapus
 20 <-- 30 <--
```

2.2.3 Class ReverseData_2511532028

```
package pekan4_2511532028;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511532028 {
    public static void main(String[] args) {
        Queue<Integer> q_2028 = new LinkedList<Integer>();
        q_2028.add(1);
        q_2028.add(2);
        q_2028.add(3);
        System.out.println("Sebelum rever : " + q_2028);
        Stack<Integer> s_2028 = new Stack<Integer>();
        while(!q_2028.isEmpty()) {
            s_2028.push(q_2028.remove());
        }
        while(!s_2028.isEmpty()) {
            q_2028.add(s_2028.pop());
        }
        System.out.println("Sesudah reverse : " + q_2028);
    }
}
```

Penjelasan :

```
Queue<Integer> q_2028 = new LinkedList<Integer>();
```

Membuat *Queue* bertipe *Integer* menggunakan *LinkedList* sebagai implementasi.

```
q_2028.add(1);  
q_2028.add(2);  
q_2028.add(3);
```

Menambahkan elemen ke dalam *Queue*.

```
System.out.println("Sebelum rever : " + q_2028);
```

Menampilkan isi queue sebelum dilakukan proses pembalikan.

```
Stack<Integer> s_2028 = new Stack<Integer>();
```

Membuat *stack* yang akan digunakan sebagai alat bantu untuk membalik urutan data.

```
while(!q_2028.isEmpty()) {  
    s_2028.push(q_2028.remove());  
}
```

Semua elemen dalam queue dipindahkan ke *stack*. *q.remove()*, untuk ambil dari depan *queue*, *s.push()*, untuk masuk ke *stack*. Karena stack bersifat *LIFO*, urutan data otomatis terbalik.

```
while(!s_2028.isEmpty()) {  
    q_2028.add(s_2028.pop());  
}
```

Data dari *stack* dikembalikan ke *queue*. *s.pop()*, untuk ambil dari atas *stack*, *q.add()*, untuk masuk ke *queue*. Hasilnya, urutan *queue* menjadi terbalik dari sebelumnya.

```
System.out.println("Sesudah reverse : " + q_2028);  
}
```

Menampilkan hasil akhir queue setelah dilakukan reverse.

Program ini menunjukkan bahwa pembalikan data pada queue dapat dilakukan dengan bantuan stack. Dengan memanfaatkan perbedaan prinsip FIFO pada queue dan LIFO pada stack, urutan elemen dalam queue dapat dibalik secara efektif.

Output :

Gambar 2.2.3

```
<terminated> ReverseData_2511532028 [Java Application]
Sebelum rever : [1, 2, 3]
Sesudah reverse : [3, 2, 1]
```

2.2.4 Class QueueLinkedList

```
package pekan4_2511532028;

import java.util.Queue;
import java.util.LinkedList;

public class QueueLinkedList_2511532028 {

    public static void main(String[] args) {
        Queue<Integer> q_2028 = new LinkedList<>();
        // tambah elemen dengan loop (0,1,2,3,4,5)
        for(int i = 0; i < 6; i++)
            q_2028.add(i);
        // Menampilkan Antrian
        System.out.println("Elemen Antian = " + q_2028);
        // menghapus antrian terdepan
        int hapus = q_2028.remove();
        System.out.println("Hapus elemen = " + hapus);
        System.out.println(q_2028);
        // melihat elemen/antrian di deoan
        int depan = q_2028.peek();
        System.out.println("Kepala Antrian = " + depan);

        int banyak = q_2028.size();
        System.out.println("Size Antrian = " + banyak);
    }
}
```

Penjelasan :

```
Queue<Integer> q_2028 = new LinkedList<>();
```

Membuat *Queue* menggunakan *LinkedList*.

```
for(int i = 0; i < 6; i++)
    q_2028.add(i);
```

Menambahkan elemen kedalam *Queue* menggunakan perulangan *for* dengan *i* sebagai index pada *Queue*.

```
System.out.println("Elemen Antian = " + q_2028);
```

Menampilkan seluruh isi *Queue* tanpa harus menggunakan perulangan *for*, karena menggunakan *LinkedList*.

```
int hapus = q_2028.remove();
```

Method *remove()* digunakan untuk menghapus elemen paling depan dari *Queue* (*FIFO*). Nilai yang dihapus disimpan dalam variabel *hapus* lalu ditampilkan.

```
int depan = q_2028.peek();
System.out.println("Kepala Antrian = " + depan);
```

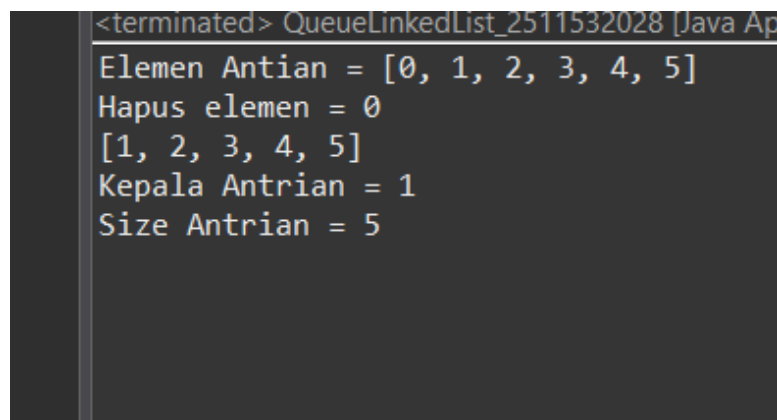
Method *peek()* digunakan untuk melihat elemen paling depan tanpa menghapusnya.

```
int banyak = q_2028.size();
System.out.println("Size Antrian = " + banyak);
```

Method *size()* digunakan untuk mengetahui jumlah elemen yang ada di dalam queue saat ini.

Output :

Gambar 2.2.4



```
<terminated> QueueLinkedList_2511532028 [Java Ap
Elemen Antian = [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

2.2.5 Class IteratorQueue_2511532028

```
package pekan4_2511532028;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Iterator;
```

```

public class IteratorQueue_2511532028 {

    public static void main(String[] args) {
        Queue<String> q_2028 = new LinkedList<>();

        q_2028.add("Praktikum");
        q_2028.add("Struktur");
        q_2028.add("Data");
        q_2028.add("Dan");
        q_2028.add("Algoritma");
        Iterator<String> iterator_2028 = q_2028.iterator();
        while (iterator_2028.hasNext()) {
            System.out.print(iterator_2028.next() + " ");
        }
    }
}

```

Penjelasan :

```

Queue<String> q_2028 = new LinkedList<>();

```

Membuat objek *Queue* menggunakan *LinkedList*.

```

q_2028.add("Praktikum");
q_2028.add("Struktur");
q_2028.add("Data");
q_2028.add("Dan");
q_2028.add("Algoritma");

```

Menambahkan beberapa elemen *String* ke dalam *Queue*. Elemen-elemen ini akan tersimpan sesuai urutan penambahan (*FIFO*)

```

Iterator<String> iterator_2028 = q_2028.iterator();

```

Membuat objek *Iterator* untuk mengakses elemen-elemen dalam *Queue*. *Iterator* digunakan untuk menelusuri isi struktur data tanpa harus menggunakan indeks.

```

while (iterator_2028.hasNext()) {
    System.out.print(iterator_2028.next() + " ");
}

```

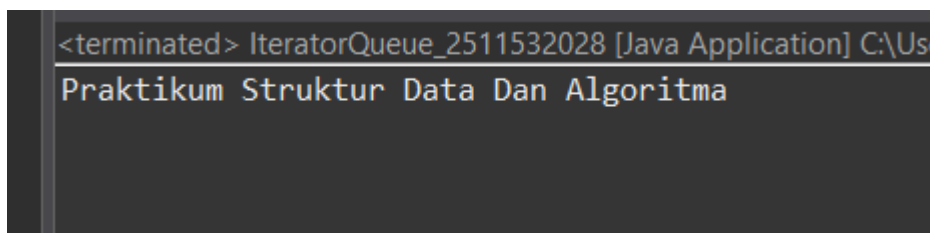
Perulangan ini digunakan untuk menampilkan semua elemen dalam *Queue*. *hasNext()*, untuk mengecek apakah masih ada elemen berikutnya, *next()*, untuk

mengambil elemen berikutnya. Setiap elemen akan dicetak hingga seluruh isi queue selesai ditampilkan

Program ini menunjukkan cara mengakses seluruh elemen dalam queue menggunakan Iterator. Dengan metode ini, elemen dapat ditelusuri secara berurutan tanpa perlu menghapus data dari queue, sehingga struktur data tetap utuh.

Output :

Gambar 2.2.5



2.3 Tugas Praktikum

2.3.1 Class TugasQueue_2511532028

```
package pekan4_2511532028;

import java.util.Scanner;

public class TugasQueue_2511532028 {
    Scanner i = new Scanner(System.in);
    int front_2028, rear_2028, size_2028;
    int kapasitas_2028;
    String antri_2028[];
    // String pelanggan_2028;

    public TugasQueue_2511532028(int kapasitas) {
        this.kapasitas_2028 = kapasitas;
        front_2028 = this.size_2028 = 0;
        rear_2028 = kapasitas - 1;
        antri_2028 = new String[this.kapasitas_2028];
    }

    public static void MenuAntri_2028() {
        System.out.println("\n====Menu Antrian Loker Pelayanan====");
        System.out.println("1. Tambah Pelanggan");
    }
}
```



```

        System.out.println("2. Hapus Pelanggan");
        System.out.println("3. Reverse Antrian Pelanggan");
        System.out.println("4. Tampilkan Antrian");
        System.out.println("5. Keluar");
    }

    boolean isEmpty(TugasQueue_2511532028 q) {
        return (q.size_2028 == 0);
    }

    boolean isFull(TugasQueue_2511532028 q) {
        return (q.size_2028 == q.kapasitas_2028);
    }

    public void enqueue_2028(Scanner i) {
        if(isFull(this)) {
            System.out.println("Antrian Penuh");
            return;
        }
        this.rear_2028 = (this.rear_2028 + 1) % this.kapasitas_2028;
        System.out.print("Masukkan nama pelanggan : ");
        String pelanggan_2028 = i.nextLine();
        this.antri_2028[this.rear_2028] = pelanggan_2028;
        this.size_2028 = this.size_2028 + 1;
        System.out.println(pelanggan_2028 + " berhasil di tambahkan");
    }

    public String dequeue_2028() {
        if(isEmpty(this)) {
            System.out.println("Antrian Kosong");
            return null;
        }
        String pelanggan_2028 = this.antri_2028[this.front_2028];
        this.front_2028 = (this.front_2028 + 1) % this.kapasitas_2028; //
Circular Queue : saat front == kapasitas, front balik ke index 0
        this.size_2028 = this.size_2028 - 1;
        System.out.println(pelanggan_2028 + " sudah diberikan pelayanan
loket");
        return pelanggan_2028;
    }

```

```

    public void reverseAntri_2028() {
        System.out.println("====Hasil Reverse Antrian====");
        for(int i = 0; i < this.size_2028; i++) {
            if (i == (this.size_2028/2))
                break;
            String temp = this.antri_2028[this.front_2028+i];
            this.antri_2028[this.front_2028+i] =
this.antri_2028[this.rear_2028-i];
            this.antri_2028[this.rear_2028-i] = temp;

            //System.out.print((hasil_2028) + " ");
        }
        System.out.print("[ ");
        for(int i = 0; i < this.size_2028; i++) {
            System.out.print(" " + this.antri_2028[this.front_2028+i] + "
");
        }
        System.out.print(" ]");
        System.out.println();
    }

    public void Tampilkan_2028() {
        System.out.println("====Tampilan dari Antrian Locket====");
        System.out.print("[ ");
        for(int i = 0; i < this.size_2028; i++) {
            System.out.print(this.antri_2028[this.front_2028 + i] + " ");
        }
        System.out.print(" ]");
        System.out.println();
    }
}

```

Penjelasan :

```

public TugasQueue_2511532028(int kapasitas) {
    this.kapasitas_2028 = kapasitas;
}

```

```

front_2028 = this.size_2028 = 0;
rear_2028 = kapasitas - 1;
antri_2028 = new String[this.kapasitas_2028];
}

```

- Menginisialisasi queue dengan kapasitas tertentu
- front = 0 dan size = 0 menandakan queue kosong
- rear = kapasitas - 1 digunakan untuk konsep **circular queue**
- Array dibuat sesuai kapasitas

```

public static void MenuAntri_2028() {
    System.out.println("\n====Menu Antrian Loker Pelayanan====");
    System.out.println("1. Tambah Pelanggan");
    System.out.println("2. Hapus Pelanggan");
    System.out.println("3. Reverse Antrian Pelanggan");
    System.out.println("4. Tampilkan Antrian");
    System.out.println("5. Keluar");
}

```

- Menampilkan menu pilihan program
- Berisi opsi tambah, hapus, reverse, tampilkan, dan keluar

```

boolean isEmpty(TugasQueue_2511532028 q) {
    return (q.size_2028 == 0);
}

boolean isFull(TugasQueue_2511532028 q) {
    return (q.size_2028 == q.kapasitas_2028);
}

```

- Mengecek apakah queue penuh atau kosong
- Jika size == kapasitas, maka queue penuh. Jika size == 0, maka queue kosong

```

public void enqueue_2028(Scanner i) {
    if(isFull(this)) {
        System.out.println("Antrian Penuh");
        return;
    }
    this.rear_2028 = (this.rear_2028 + 1) % this.kapasitas_2028;
    System.out.print("Masukkan nama pelanggan : ");
    String pelanggan_2028 = i.nextLine();
    this.antri_2028[this.rear_2028] = pelanggan_2028;
    this.size_2028 = this.size_2028 + 1;
}

```

```

        System.out.println(pelanggan_2028 + " berhasil di tambahkan");
    }

```

- Mengecek apakah queue penuh , Jika tidak
- Menggunakan **circular queue**, jika sudah di ujung akan kembali ke awal
- Input nama pelanggan
- Data disimpan ke array
- Size bertambah
- Menampilkan pesan berhasil

```

public String dequeue_2028() {
    if(isEmpty(this)) {
        System.out.println("Antrian Kosong");
        return null;
    }
    String pelanggan_2028 = this.antri_2028[this.front_2028];
    this.front_2028 = (this.front_2028 + 1) % this.kapasitas_2028; //
Circular Queue : saat front == kapasitas, front balik ke index 0
    this.size_2028 = this.size_2028 - 1;
    System.out.println(pelanggan_2028 + " sudah diberikan pelayanan
loket");
    return pelanggan_2028;
}

```

- Mengecek apakah queue kosong
- Mengambil data di posisi front
- Menggeser front dan menggunakan **circular queue**
- Mengurangi size
- Menampilkan pelanggan yang sudah dilayani

```

public void reverseAntri_2028() {
    System.out.println("====Hasil Reverse Antrian====");
    for(int i = 0; i < this.size_2028; i++) {
        if (i == (this.size_2028/2))
            break;
        String temp = this.antri_2028[this.front_2028+i];
        this.antri_2028[this.front_2028+i] =
this.antri_2028[this.rear_2028-i];
        this.antri_2028[this.rear_2028-i] = temp;

        //System.out.print((hasil_2028) + " ");
    }
}

```

```

    }
    System.out.print("[ ");
    for(int i = 0; i < this.size_2028; i++) {
        System.out.print(" " + this.antri_2028[this.front_2028+i] + "
");
    }
    System.out.print(" ]");
    System.out.println();
}

```

- Membalik urutan antrian tanpa stack
- Menggunakan teknik **swap** antara depan dan belakang
- Perulangan hanya sampai setengah ($\text{size}/2$), jadi misalkan sizenya atau isi datanya ada 5 atau berjumlah ganjil. Jadi saat dijalankan reverse ini, perulangannya sampe index ke 2, saat index ke 3 perulangan berhenti karena data ke 3 dari 5 data tersebut posisinya tetap walaupun di balik. Jika perulangannya sebanyak ($\leq \text{size}$) maka reverse tidak ada artinya. Yang awal datanya (adi , rendi, budi) tetap menjadi (adi, rendi, budi) karena menggunakan Teknik swap tersebut.
- Setelah itu ditampilkan hasil antrian yang sudah dibalik

```

public void Tampilkan_2028() {
    System.out.println("====Tampilan dari Antrian Locket====");
    System.out.print("[ ");
    for(int i = 0; i < this.size_2028; i++) {
        System.out.print(this.antri_2028[this.front_2028 + i] + " ");
    }
    System.out.print(" ]");
    System.out.println();
}

```

- Menampilkan seluruh isi queue
- Menggunakan perulangan dari front sebanyak size
- Format output berbentuk list

2.3.2 Class AntrianLoket_2511532028

```

package pekan4_2511532028;

import java.util.Scanner;

```

```

public class AntrianLoket_2511532028 {

    public static void main(String[] args) {
        Scanner z = new Scanner(System.in);
        TugasQueue_2511532028 q_2028 = new TugasQueue_2511532028(5);
        int pilihan_2028;
        do {
            q_2028.MenuAntri_2028();
            System.out.print("Pilih menu : ");
            pilihan_2028 = z.nextInt();
            System.out.println();
            z.nextLine();

            switch (pilihan_2028) {
                case 1:
                    q_2028.enqueue_2028(z);
                    break;
                case 2:
                    q_2028.dequeue_2028();
                    break;
                case 3:
                    q_2028.reverseAntri_2028();
                    break;
                case 4:
                    q_2028.Tampilkan_2028();
                    break;
                case 5:
                    System.out.println("===Anda Keluar dari Menu
Antrian===");
                    break;
            }
        } while (pilihan_2028 != 5);
        z.close();
    }
}

```

Penjelasan :

```
TugasQueue_2511532028 q_2028 = new TugasQueue_2511532028(5);
```

- Membuat objek queue dengan kapasitas maksimal 5 data
- Queue ini akan menyimpan nama pelanggan

```
int pilihan_2028;
```

Digunakan untuk menyimpan pilihan menu.

```
do {  
    q_2028.MenuAntri_2028();  
    System.out.print("Pilih menu : ");  
    pilihan_2028 = z.nextInt();  
    System.out.println();  
    z.nextLine();  
  
    switch (pilihan_2028) {  
        case 1:  
            q_2028.enqueue_2028(z);;  
            break;  
        case 2:  
            q_2028.dequeue_2028();  
            break;  
        case 3:  
            q_2028.reverseAntri_2028();;  
            break;  
        case 4:  
            q_2028.Tampilkan_2028();  
            break;  
        case 5:  
            System.out.println("===Anda Keluar dari Menu  
Antrian===");  
            break;  
    }  
} while (pilihan_2028 != 5);  
z.close();
```

- Digunakan agar program terus berjalan sesuai apa yang dipilih sampai user memilih keluar.
- Program akan berhenti jika memilih menu 5

Output :

Gambar 2.3.2

Menambahkan data hingga penuh :

```
AntrianLoket_2511532028 [Java Application] C:\Users\USER\p2\pool\plugin
=====Menu Antrian Locket Pelayanan=====
1. Tambah Pelanggan
2. Hapus Pelanggan
3. Reverse Antrian Pelanggan
4. Tampilkan Antrian
5. Keluar
Pilih menu : 1
Masukkan nama pelanggan : siti
siti berhasil di tambahkan

=====Menu Antrian Locket Pelayanan=====
1. Tambah Pelanggan
2. Hapus Pelanggan
3. Reverse Antrian Pelanggan
4. Tampilkan Antrian
5. Keluar
Pilih menu : 1

Antrian Penuh

=====Menu Antrian Locket Pelayanan=====
1. Tambah Pelanggan
2. Hapus Pelanggan
3. Reverse Antrian Pelanggan
4. Tampilkan Antrian
5. Keluar
Pilih menu : 4

=====Tampilan dari Antrian Locket=====
[ rani toni budi revi siti ]
```

Membalikkan antrian :

```
8ja
=====Tampilan dari Antrian Locket=====
[ rani toni budi revi siti ]

=====Menu Antrian Locket Pelayanan=====
1. Tambah Pelanggan
2. Hapus Pelanggan
3. Reverse Antrian Pelanggan
4. Tampilkan Antrian
5. Keluar
Pilih menu : 3
|
=====Hasil Reverse Antrian=====
[ siti revi budi toni rani ]
```

Menghapus data yang sudah dibalikkan sebelumnya :


```

=====Menu Antrian Locket Pelayanan=====
1. Tambah Pelanggan
2. Hapus Pelanggan
3. Reverse Antrian Pelanggan
4. Tampilkan Antrian
5. Keluar
Pilih menu : 2

siti sudah diberikan pelayanan loket

=====Menu Antrian Locket Pelayanan=====
1. Tambah Pelanggan
2. Hapus Pelanggan
3. Reverse Antrian Pelanggan
4. Tampilkan Antrian
5. Keluar
Pilih menu : 2

revi sudah diberikan pelayanan loket

=====Menu Antrian Locket Pelayanan=====
1. Tambah Pelanggan
2. Hapus Pelanggan
3. Reverse Antrian Pelanggan
4. Tampilkan Antrian
5. Keluar
Pilih menu : 4

====Tampilan dari Antrian Locket====
[ budi toni rani ]

```

Saat dihapus hingga data di dalam Queue kosong :

```

AntrianLocket_2511532028 [Java Application] C:\Users\USER\p2\
2. Hapus Pelanggan
3. Reverse Antrian Pelanggan
4. Tampilkan Antrian
5. Keluar
Pilih menu : 2

toni sudah diberikan pelayanan loket

=====Menu Antrian Locket Pelayanan=====
1. Tambah Pelanggan
2. Hapus Pelanggan
3. Reverse Antrian Pelanggan
4. Tampilkan Antrian
5. Keluar
Pilih menu : 2

rani sudah diberikan pelayanan loket

=====Menu Antrian Locket Pelayanan=====
1. Tambah Pelanggan
2. Hapus Pelanggan
3. Reverse Antrian Pelanggan
4. Tampilkan Antrian
5. Keluar
Pilih menu : 2

Antrian Kosong

```

Program AntrianLoket_2511532028 merupakan program utama (driver) yang digunakan untuk menjalankan sistem antrian loket. Program ini menyediakan menu interaktif yang memungkinkan pengguna untuk:

- Menambahkan pelanggan ke dalam antrian
- Menghapus pelanggan yang telah dilayani
- Membalik urutan antrian
- Menampilkan isi antrian

Program ini memanfaatkan class TugasQueue_2511532028 sebagai implementasi queue dengan konsep FIFO, sehingga dapat mensimulasikan sistem antrian secara sederhana dan interaktif.

BAB III

PENUTUP

3.1 Kesimpulan

Queue merupakan struktur data yang menggunakan prinsip FIFO, di mana elemen pertama yang masuk akan menjadi yang pertama keluar. Implementasi queue dapat dilakukan menggunakan array maupun LinkedList.

Melalui praktikum ini, dapat dipahami bagaimana operasi dasar queue bekerja serta bagaimana penerapannya dalam kehidupan nyata seperti sistem antrian.

3.2 Saran

Dalam mempelajari *Queue* dan *LinkedList*, disarankan untuk memahami konsep dasar terlebih dahulu sebelum mengimplementasikan program. Latihan yang beragam akan membantu meningkatkan pemahaman terhadap struktur data ini.

Referensi

Data Structures and Algorithms in Java

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th Edition). Wiley.

Oracle. *Java Collections Framework*.